

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática
Especialidad de Computación

SimAS 3.0 descendente predictivo

Simulador de analizadores sintácticos descendentes predictivos

22 de septiembre de 2025



- Título: SimAS 3.0 descendente predictivo. Simulador de analizadores sintácticos descendentes predictivos.
- Autor: D. Antonio Llamas García
- Directores: Prof. Dr. Antonio Araúzo Azofra; Prof. Dra. María Luque Rodríguez
- Centro: Escuela Politécnica Superior de la Universidad de Córdoba

Introducción

Objetivos

Antecedentes

Requisitos

Diseño

Pruebas

Demostración del software

Conclusiones

- SimAS 3.0 es un simulador educativo para comprender cómo funciona el análisis sintáctico descendente predictivo (LL(1)).
- Permite crear y editar gramáticas de contexto libre, construir la tabla predictiva y seguir, paso a paso, la derivación y el árbol sintáctico generado.
- La aplicación ha evolucionado desde SimAS 1.0 y 2.0, con una interfaz renovada, pestañas inteligentes, generación de informes y soporte multiidioma.

- Se partía de una versión previa con problemas de consistencia y portabilidad. Esta versión los corrige y unifica la experiencia para que no se pierda información al navegar ni al guardar.
- El trabajo se centra en el análisis descendente predictivo: eliminación de recursividad y factorización cuando procede, cálculo fiable de Primero/Siguiente y construcción de la tabla predictiva $LL(1)$.
- Además, se incorporan funciones de recuperación de errores, informes completos y compatibilidad en Windows, macOS y Linux.

- Una forma visual y dinámica de aprender LL(1): la derivación y el árbol se construyen en tiempo real.
- Un asistente guiado que acompaña los pasos clave (transformaciones, conjuntos y tabla) con explicaciones y representación clara.
- Una base estable y coherente para la docencia: interfaz limpia, pestañas para trabajar con varias vistas y documentación exportable.

Contenido

Introducción

Objetivos

Antecedentes

Requisitos

Diseño

Pruebas

Demostración del software

Conclusiones

Objetivo general

- Construir una herramienta pedagógica sólida para entender y practicar el análisis sintáctico descendente predictivo (LL(1)), uniendo teoría y visualización paso a paso.

Objetivos específicos

- Robustecer el flujo completo: transformaciones de gramáticas cuando proceda, cálculo correcto de Primero/Siguiente y construcción de la tabla predictiva LL(1) sin ambigüedades.
- Mejorar la experiencia de aprendizaje con un asistente claro y con derivación/árbol en tiempo real.
- Facilitar la práctica: edición de gramáticas, funciones de recuperación de errores y generación de informes.

El proyecto persigue una mejora tangible en la calidad del software:

- **Consistencia:** datos y estado coherentes al navegar y al guardar.
- **Eficiencia:** cálculos y renderizados ágiles en equipos estándar.
- **Fiabilidad:** resultados deterministas y manejo sólido de errores.
- **Usabilidad:** interfaz limpia, asistente por pasos y mensajes claros.
- **Compatibilidad:** correcto funcionamiento con recursos y formatos esperados.
- **Portabilidad:** funcionamiento equivalente en Windows, macOS y Linux.
- **Mantenibilidad:** código modular, patrones conocidos y documentación.
- **Extensibilidad:** base preparada para nuevas funciones y vistas.

Resultados esperados

- Experiencias de simulación consistentes, sin pérdidas de datos al navegar entre vistas.
- Portabilidad real (Windows, macOS y Linux) y una interfaz limpia basada en pestañas.
- Un soporte didáctico reutilizable en clase y para el autoestudio (PDF/HTML).

Introducción

Objetivos

Antecedentes

Requisitos

Diseño

Pruebas

Demostración del software

Conclusiones

- SimAS 1.0 sentó las bases: editor de gramáticas y simulación de analizadores descendentes y ascendentes.
- SimAS 2.0 amplió funciones: validación automática, más control sobre tablas y errores, e informes; pero aparecieron problemas de consistencia y portabilidad.

- La docencia necesita una herramienta estable: sin pérdidas de estado ni resultados inconsistentes entre pasos.
- La visualización es clave: derivación y árbol ayudan a fijar conceptos, pero deben generarse de forma fiable.
- La experiencia debe ser fluida: trabajar con varias vistas/pestañas y generar informes de forma sencilla.

Qué aporta SimAS 3.0 respecto a 2.0

- Correcciones en transformaciones, conjuntos Primero/Siguiente y construcción de la tabla predictiva LL(1).
- Derivación y árbol en tiempo real, incluyendo nodos ϵ , y gestión clara de errores.
- Interfaz con pestañas inteligentes, i18n y ejecución consistente en Windows, macOS y Linux.

Introducción

Objetivos

Antecedentes

Requisitos

Diseño

Pruebas

Demostración del software

Conclusiones

- El sistema se organiza en cuatro bloques: editor de gramáticas, simulador LL(1), tutorial y ayuda.
- La información fluye desde la edición hasta la simulación, manteniendo siempre la gramática y los artefactos generados.

- **Editor:** crear/cargar/guardar gramáticas; validar símbolos, producciones y símbolo inicial; consultar y generar informe de gramática.
- **Simulador:** Primero/Siguiente, tabla predictiva LL(1), funciones de recuperación de errores, simulación continua y paso a paso, derivación y árbol.
- **Documentación:** informes (PDF) y recursos de ayuda; tutorial accesible desde cualquier vista.

- **Usabilidad:** interfaz clara, asistente por pasos y mensajes comprensibles.
- **Fiabilidad:** resultados deterministas y control de errores en todas las operaciones.
- **Compatibilidad/Portabilidad:** funcionamiento equivalente en Windows, macOS y Linux.
- **Rendimiento:** respuesta fluida en cálculos y renderizado del árbol/derivación.
- **Mantenibilidad/Extensibilidad:** diseño modular con patrones (MVC, Strategy, Observer) y recursos i18n.

- Interfaz basada en pestañas con navegación padre–hijo y estados consistentes.
- Gramática estructurada (símbolos, producciones, símbolo inicial) y artefactos derivados (Primero/Siguiente, tabla, informe, derivación y árbol).

Introducción

Objetivos

Antecedentes

Requisitos

Diseño

Pruebas

Demostración del software

Conclusiones

- Arquitectura modular con separación clara entre *modelo*, *lógica de análisis* y *presentación*.
- Persistencia de artefactos de gramática para mantener consistencia: gramática, Primero/Siguiente, tabla predictiva LL(1), derivación y árbol.
- Interfaz basada en pestañas inteligentes (padre-hijo) para trabajar en paralelo sin perder el contexto.

Grupos de gramáticas y dependencias

- Cada **editor** abre una gramática y forma un *grupo lógico* con los **simuladores** lanzados desde él: comparten gramática y artefactos (Primero/Siguiente, tabla LL(1), derivación y árbol).
- Las dependencias se mantienen dentro del grupo: cambios en el editor se reflejan en los simuladores del mismo grupo de forma coherente.
- También es posible abrir **simuladores independientes** (sin editor activo). En ese caso, el simulador trabaja con una gramática seleccionada/importada sin crear dependencia con ningún editor.

- **bienvenida**: pantalla inicial y acceso rápido a tareas frecuentes.
- **gramatica**: modelo de gramática y artefactos derivados (Primero/Siguiente, tabla LL(1), árbol, derivación).
- **editor**: edición/validación de gramáticas e informes.
- **simulador**: ejecución paso a paso/continua y visualización.
- **utils**: utilidades comunes y servicios de apoyo.
- **resources**: internacionalización (i18n), textos y recursos gráficos.

- MVC para desacoplar modelo, lógica y vistas.
- Strategy para algoritmos de análisis y recuperación de errores.
- Observer para sincronizar vistas con cambios en el modelo.
- Principios SOLID para facilitar mantenibilidad y extensibilidad.

- Flujo guiado: edición → análisis → simulación → informe.
- **Grupos de gramáticas:** pestañas padre-hijo con gramática compartida, manteniendo sincronía de datos.
- **Simuladores independientes:** pueden abrirse sin editor; operan sobre una gramática elegida sin depender de una pestaña de edición.
- Vistas sincronizadas: cualquier cambio en la gramática del grupo actualiza artefactos y simulación.
- Internacionalización: recursos de texto separados y conmutables en tiempo de ejecución.

Contenido

Introducción

Objetivos

Antecedentes

Requisitos

Diseño

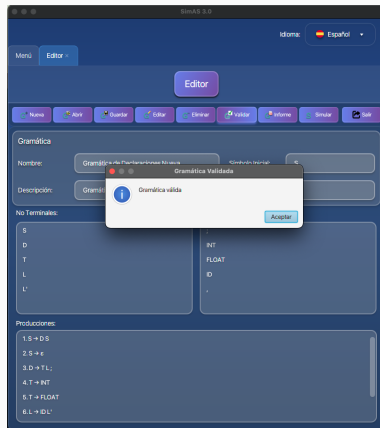
Pruebas

Demostración del software

Conclusiones

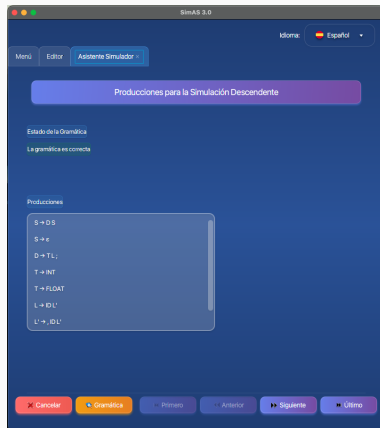
Pruebas: validación de gramática

Validación coherente de símbolos, producciones y símbolo inicial.



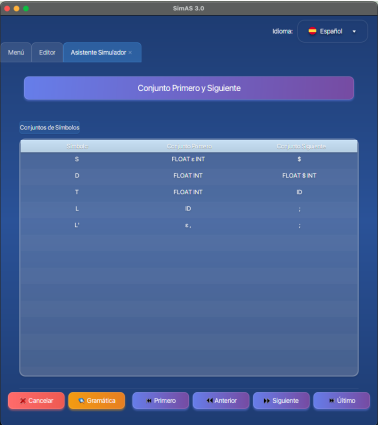
Pruebas: recursividad y factorización

Eliminación de recursividad izquierda y factorización correctas.



Pruebas: conjuntos Primero/Siguiente

Cálculo estable y correcto de los conjuntos Primero/Siguiente.



SimAS 3.0

Idioma: 🇪🇸 Español

Menú Editor Asistente Simulador >

Conjunto Primero y Siguiente

Conjuntos de Símbolos

Símbolo	Conjunto Primero	Conjunto Siguiente
S	Float e INT	\$
D	Float INT	Float \$ INT
T	Float INT	ID
L	ID	:
L'	ε	:

Cancelar Gramática Primero Anterior Siguiente Último

Pruebas: tabla predictiva LL(1)

Construcción de la tabla predictiva LL(1) sin ambigüedades.

SimAS 3.0

Idioma: 🇪🇸 Español

Menú Editor Asistente Simulador <

Tabla Simulación

Seleccione una función de error

3 - Insertar un Símbolo en la Entrada: INT

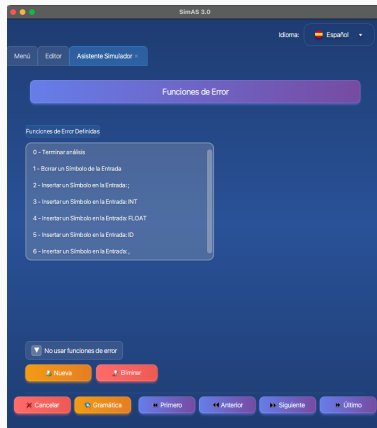
No. ítem	S	INT	FLOAT	ID	;	\$
S	ϵ	$1.S \rightarrow D.S$	$1.S \rightarrow D.S$	ϵ	ϵ	$2.S \rightarrow \epsilon$
D	ϵ	$3.D \rightarrow T.L;$	$3.D \rightarrow T.L;$	ϵ	ϵ	ϵ
T	ϵ	$4.T \rightarrow INT$	$5.T \rightarrow FLOAT$	ϵ	ϵ	ϵ
L	ϵ	E0	ϵ	$6.L \rightarrow ID.L'$	ϵ	ϵ
L'	$8.L' \rightarrow \epsilon$	ϵ	E0	ϵ	$7.L' \rightarrow ;.ID.L'$	ϵ
;	ϵ	ϵ	ϵ	E0	ϵ	ϵ
INT						
FLOAT						
ID	ϵ	ϵ	ϵ	ϵ	ϵ	ϵ
;						
\$	ϵ	ϵ	ϵ	E3	ϵ	ϵ

Borrar Rollear Reiniciar

Cancelar Gramática Primero Anterior Siguiente Simulación

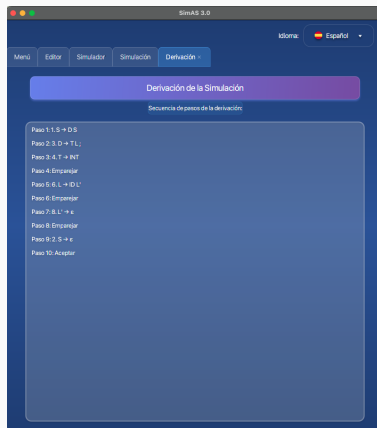
Pruebas: funciones de recuperación de errores

Integración de funciones de error en la construcción y simulación.



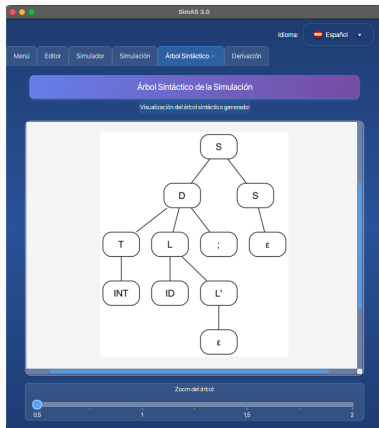
Pruebas: derivación paso a paso

Simulación determinista de la derivación de la cadena.



Pruebas: árbol sintáctico

Construcción del árbol sintáctico consistente con la derivación.



Introducción

Objetivos

Antecedentes

Requisitos

Diseño

Pruebas

Demostración del software

Conclusiones

Demostración del software

Contenido

Introducción

Objetivos

Antecedentes

Requisitos

Diseño

Pruebas

Demostración del software

Conclusiones

- SimAS 3.0 evoluciona la herramienta a un entorno pedagógico moderno centrado en análisis descendente predictivo.
- Se alcanza consistencia total: sin pérdidas de información entre módulos y resultados deterministas.
- Eficiencia y experiencia: menor consumo, respuesta ágil y navegación por pestañas más productiva.

- Correcciones clave: Primero/Siguiente y tabla predictiva LL(1) correctas; recursividad y factorización arregladas; informes fiables.
- Sistema inteligente de navegación por pestañas que permite trabajar con varias gramáticas simultáneamente.
- Internacionalización dinámica y arquitectura modular (principios SOLID) para mantener y extender la aplicación.